

PYTHON PROGRAMMING INTERNSHIP

PROJECT 1 (ADVANCED EDITION) To-Do List Manager — Professional GUI Application

Built with Tkinter, JSON Persistence, and OOP Architecture

Industrial Training Kit | Powered by DecodeLabs
Batch: 2026

Prepared by: Muheeb-ur-Rehman
Role: Python Developer Intern

1. Overview: From CLI to Professional GUI

The original Project 1 deliverable was a command-line To-Do List Manager. This advanced edition upgrades the project into a professional, resume-worthy desktop application using Python's built-in Tkinter library, while preserving full compatibility with the original learning objectives (lists, functions, loops, and the IPO model).

This version introduces software engineering practices used in real production code: object-oriented design (classes), separation of data logic from interface logic, persistent storage with JSON, and a modern dark-themed user interface.

1.1 What's New in the Advanced Version

Feature	Original CLI Version	Advanced GUI Version
Interface	Text menu in terminal	Modern dark-themed desktop window (Tkinter)
Data Storage	In-memory list (lost on exit)	Persistent JSON file (tasks.json) — survives restarts
Add Task	Type number, then text	Type text, press Enter or click 'Add Task'
View Tasks	Numbered text printout	Scrollable list with checkboxes-style markers
Mark Complete	Not available	Double-click or 'Mark Done / Undo' button
Edit Task	Not available	'Edit' button opens an edit dialog
Delete Task	Not available	'Delete' button with confirmation prompt
Bulk Cleanup	Not available	'Clear Completed' button removes all done tasks
Code Structure	Procedural functions	Object-Oriented (TaskManager + TodoApp classes)
Status Display	Printed total count	Live status bar: Total / Pending / Completed

1.2 Why This Architecture Matters

The advanced version separates the application into two classes, mirroring the Model-View-Controller pattern introduced in the original training material:

- TaskManager (Model): Owns the data. Handles loading from and saving to tasks.json, and all add/edit/delete/toggle logic. Has zero knowledge of the GUI.
- TodoApp (View + Controller): Owns the window, widgets, colors, and fonts. Responds to button clicks and keyboard events, and tells TaskManager what to do.

Why separate these? Because TaskManager could be reused as-is in a completely different interface — a web app, a mobile app, or a command-line tool — without changing a single line of its logic. This is a core principle of professional software design called 'separation of concerns.'

2. New Concepts Introduced in the Advanced Version

2.1 Object-Oriented Programming (Classes)

A class is a blueprint for creating objects that bundle data (attributes) and behavior (methods) together. Think of a class like a cookie cutter, and each object created from it (an 'instance') is an individual cookie — same shape, but each can have different toppings (data).

```
class TaskManager:
    def __init__(self, filepath=DATA_FILE):
        self.filepath = filepath      # an attribute (data)
        self.tasks = self._load_tasks() # an attribute (data)

    def add_task(self, text):          # a method (behavior)
        ...

# Creating an object (instance) from the class:
manager = TaskManager()
manager.add_task("Buy Milk")
```

`self` refers to 'this specific object.' Every method inside a class receives `self` as its first parameter automatically — it's how an object refers to its own data.

2.2 Dictionaries for Structured Task Data

Instead of storing each task as plain text, every task is now a dictionary — a collection of labeled key-value pairs. This matches exactly what your original training material called 'the in-memory database' (Dictionary -> Table Row).

```
task = {
    "text": "Buy Milk",
    "done": False,
    "created": "2026-06-13 10:30"
}

# Accessing values:
print(task["text"])    # Output: Buy Milk
print(task["done"])    # Output: False
```

2.3 JSON Persistence (Solving 'The Volatile Trap')

Your original PDF warned: 'RAM is volatile. Process Terminated = Data Lost.' The advanced version solves this exact problem using the `json` module — exactly as described in the 'Engineering Requires Persistence' slide of your training kit.

```
import json

# SAVE: convert Python list of dicts -> JSON text -> write to disk
with open("tasks.json", "w") as f:
```

```

    json.dump(tasks, f, indent=4)

# LOAD: read JSON text from disk -> convert back to Python list
with open("tasks.json", "r") as f:
    tasks = json.load(f)

```

This means: close the app, reopen it tomorrow, and all your tasks are still there. This is the leap from 'RAM (Variables)' to 'DISK (Files/DB)' shown in your training material.

2.4 Tkinter — Python's Built-In GUI Toolkit

Tkinter is included with every standard Python installation — no extra installation needed. Key widgets used in this project:

Widget	Purpose in This App
Tk()	The main application window
Frame	An invisible container used to group and organize other widgets
Label	Displays static text (titles, status messages)
Entry	A single-line text box for typing a new task
Button	A clickable button that triggers a function (Add, Edit, Delete...)
Listbox	A scrollable list displaying all tasks
Scrollbar	Lets the user scroll through a long task list
StringVar	A special variable that automatically syncs with widget content
messagebox	Pop-up dialogs for warnings, confirmations, and info
simpdialog	A pop-up dialog for getting text input (used for editing tasks)

2.5 Event Binding

Event binding connects a user action (like pressing Enter or double-clicking) to a function. It's like wiring a doorbell button to a chime — pressing the button (event) triggers the sound (function).

```

# When the user presses Enter inside the text box, run on_add_task
self.entry.bind("<Return>", lambda event: self.on_add_task())

# When the user double-clicks a task, toggle its completed status
self.listbox.bind("<Double-Button-1>", lambda event: self.on_toggle_done())

```

2.6 Lambda Functions

A lambda is a small, throwaway, unnamed function written in one line. It's used here because `bind()` needs a function that accepts an 'event' argument, but our handler methods (like `on_add_task`) don't need that argument — the lambda acts as a small adapter between them.

```

# Without lambda (would cause an error - event is passed automatically):

```

```
# self.entry.bind("<Return>", self.on_add_task)

# With lambda (correct - absorbs the unwanted 'event' argument):
self.entry.bind("<Return>", lambda event: self.on_add_task())
```

2.7 List Comprehensions

A list comprehension builds a new list by filtering or transforming an existing one, in a single readable line.

```
# Keep only tasks where 'done' is False (i.e., remove completed tasks)
self.tasks = [t for t in self.tasks if not t["done"]]

# Equivalent longer version using a regular loop:
new_tasks = []
for t in self.tasks:
    if not t["done"]:
        new_tasks.append(t)
self.tasks = new_tasks
```

2.8 The datetime Module

Used to record exactly when each task was created — a small but professional touch that adds real value (e.g., for sorting tasks by age later).

```
from datetime import datetime

timestamp = datetime.now().strftime("%Y-%m-%d %H:%M")
print(timestamp) # Example output: 2026-06-13 14:05
```

3. Application Architecture and Design

3.1 File and Class Structure

```

todo_gui.py
-- Constants (colors, fonts, file paths)

-- class TaskManager          (MODEL - data logic)
|  -- __init__()
|  -- _load_tasks()
|  -- save_tasks()
|  -- add_task()
|  -- toggle_done()
|  -- edit_task()
|  -- delete_task()
|  -- clear_completed()
|  -- counts()

-- class TodoApp              (VIEW + CONTROLLER - GUI)
|  -- __init__()
|  -- _configure_window()
|  -- _configure_fonts()
|  -- _build_layout()
|  |  -- _build_header()
|  |  -- _build_input_area()
|  |  -- _build_task_list_area()
|  |  -- _build_footer()
|
|  -- on_add_task()           <- event handlers
|  -- on_toggle_done()
|  -- on_edit_task()
|  -- on_delete_task()
|  -- on_clear_completed()
|
|  -- _get_selected_index()   <- helper methods
|  -- _prompt_edit_dialog()
|  -- refresh_task_list()
|  -- _update_status()
|
-- main()                    (entry point)
-- if __name__ == "__main__": main()

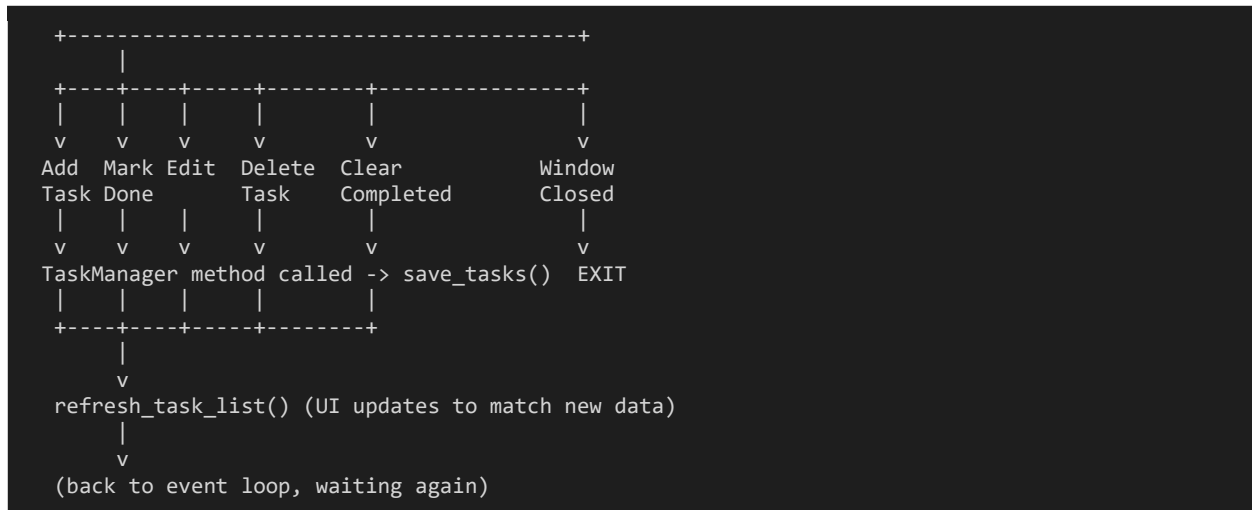
```

3.2 Application Flow Diagram

```

PROGRAM START
|
v
Create TaskManager -> load tasks.json (if it exists)
|
v
Create TodoApp -> build window, widgets, layout
|
v
refresh_task_list() -> populate Listbox + status bar
|
v
+-----+
|  EVENT LOOP (root.mainloop())  |
|  Waits for: typing, clicking, double-click |
+-----+

```



3.3 Color Palette (Modern Dark Theme)

Constant	Hex Code	Used For
COLOR_BG	#1E1E2E	Main window background
COLOR_SIDEBAR	#181825	Header and footer bars
COLOR_CARD	#2A2A3C	Input box and task list background
COLOR_ACCENT	#89B4FA	Primary buttons and highlights (blue)
COLOR_SUCCESS	#A6E3A1	'Mark Done' button (green)
COLOR_DANGER	#F38BA8	'Delete' button (red/pink)
COLOR_TEXT	#CDD6F4	Primary readable text (light lavender)
COLOR_TEXT_DIM	#9399B2	Secondary / muted text
COLOR_DONE	#6C7086	Text color for completed tasks (greyed out)

4. Complete Advanced Source Code (todo_gui.py)

This is the entire professional source file. Save it exactly as `todo_gui.py`. The program requires no installation beyond Python itself — Tkinter and json are part of the Python standard library.

```

"""
=====
PROJECT 1 (ADVANCED): TO-DO LIST MANAGER - GUI EDITION
=====
Organization : DecodeLabs Industrial Training
Batch        : 2026
Developer    : Faraz
Description   : A professional desktop To-Do List application
                  built with Tkinter. Features include adding,
                  completing, deleting, and editing tasks, task
                  counters, persistent storage using JSON, and
                  a modern, styled user interface.

Tech Stack   : Python 3.x, Tkinter (standard library),
                  JSON (standard library)
=====
"""

import json
import os
from datetime import datetime
from tkinter import (
    Tk, Frame, Label, Entry, Button, Listbox, Scrollbar,
    StringVar, messagebox, END, SINGLE, BOTH, LEFT, RIGHT, Y, X,
    TOP, BOTTOM, CENTER
)
from tkinter import font as tkfont

# -----
# CONFIGURATION CONSTANTS
# -----
DATA_FILE = "tasks.json"

COLOR_BG = "#1E1E2E"          # Main background (dark slate)
COLOR_SIDEBAR = "#181825"     # Sidebar / header background
COLOR_CARD = "#2A2A3C"       # Card / list background
COLOR_ACCENT = "#89B4FA"      # Accent blue
COLOR_ACCENT_DARK = "#74A0E0" # Pressed/hover accent
COLOR_SUCCESS = "#A6E3A1"     # Green for completed tasks
COLOR_DANGER = "#F38BA8"      # Red for delete actions
COLOR_TEXT = "#CDD6F4"        # Primary text
COLOR_TEXT_DIM = "#9399B2"    # Secondary / muted text
COLOR_DONE = "#6C7086"        # Completed task text color

FONT_FAMILY = "Segoe UI"

class TaskManager:
    """
    Handles all task DATA operations (the 'Model' layer).
    Responsible for loading, saving, and modifying the task list.
    Completely independent of the GUI - could be reused with a
    different interface (web, CLI, etc.) without changes.
    """

```

```

def __init__(self, filepath=DATA_FILE):
    self.filepath = filepath
    self.tasks = self._load_tasks()

def _load_tasks(self):
    """Load tasks from the JSON file. Return an empty list if not found."""
    if os.path.exists(self.filepath):
        try:
            with open(self.filepath, "r", encoding="utf-8") as f:
                return json.load(f)
            except (json.JSONDecodeError, IOError):
                return []
    return []

def save_tasks(self):
    """Persist the current task list to the JSON file."""
    with open(self.filepath, "w", encoding="utf-8") as f:
        json.dump(self.tasks, f, indent=4)

def add_task(self, text):
    """Add a new task dictionary to the list and save."""
    task = {
        "text": text,
        "done": False,
        "created": datetime.now().strftime("%Y-%m-%d %H:%M")
    }
    self.tasks.append(task)
    self.save_tasks()

def toggle_done(self, index):
    """Flip the completion status of a task by its index."""
    if 0 <= index < len(self.tasks):
        self.tasks[index]["done"] = not self.tasks[index]["done"]
        self.save_tasks()

def edit_task(self, index, new_text):
    """Update the text of an existing task."""
    if 0 <= index < len(self.tasks):
        self.tasks[index]["text"] = new_text
        self.save_tasks()

def delete_task(self, index):
    """Remove a task from the list by its index."""
    if 0 <= index < len(self.tasks):
        del self.tasks[index]
        self.save_tasks()

def clear_completed(self):
    """Remove all tasks marked as done."""
    self.tasks = [t for t in self.tasks if not t["done"]]
    self.save_tasks()

def counts(self):
    """Return (total, completed, pending) counts."""
    total = len(self.tasks)
    completed = sum(1 for t in self.tasks if t["done"])
    pending = total - completed
    return total, completed, pending

```

```

class TodoApp:
    """
    The main GUI application (the 'View' + 'Controller' layer).
    Builds the window, widgets, and connects user actions to the
    TaskManager (Model).
    """

    def __init__(self, root):
        self.root = root
        self.manager = TaskManager()

        self._configure_window()
        self._configure_fonts()
        self._build_layout()
        self.refresh_task_list()

    # -----
    # WINDOW SETUP
    # -----
    def _configure_window(self):
        self.root.title("DecodeLabs To-Do List Manager")
        self.root.geometry("520x640")
        self.root.minsize(440, 520)
        self.root.configure(bg=COLOR_BG)

    def _configure_fonts(self):
        self.font_title = tkfont.Font(family=FONT_FAMILY, size=20, weight="bold")
        self.font_subtitle = tkfont.Font(family=FONT_FAMILY, size=10)
        self.font_normal = tkfont.Font(family=FONT_FAMILY, size=12)
        self.font_small = tkfont.Font(family=FONT_FAMILY, size=9)
        self.font_button = tkfont.Font(family=FONT_FAMILY, size=11, weight="bold")

    # -----
    # LAYOUT CONSTRUCTION
    # -----
    def _build_layout(self):
        self._build_header()
        self._build_input_area()
        self._build_task_list_area()
        self._build_footer()

    def _build_header(self):
        header = Frame(self.root, bg=COLOR_SIDEBAR, height=90)
        header.pack(fill=X, side=TOP)

        title = Label(
            header, text="DecodeLabs To-Do List",
            font=self.font_title, fg=COLOR_TEXT, bg=COLOR_SIDEBAR
        )
        title.pack(pady=(18, 0))

        subtitle = Label(
            header, text="Python Internship | Batch 2026 | Project 1 (Advanced)",
            font=self.font_subtitle, fg=COLOR_TEXT_DIM, bg=COLOR_SIDEBAR
        )
        subtitle.pack(pady=(2, 14))

    def _build_input_area(self):
        input_frame = Frame(self.root, bg=COLOR_BG)
        input_frame.pack(fill=X, padx=20, pady=(18, 10))

```

```

self.entry_var = StringVar()
self.entry = Entry(
    input_frame, textvariable=self.entry_var,
    font=self.font_normal, bg=COLOR_CARD, fg=COLOR_TEXT,
    insertbackground=COLOR_TEXT, relief="flat", bd=0
)
self.entry.pack(side=LEFT, fill=X, expand=True, ipady=8, padx=(0, 10))
self.entry.bind("<Return>", lambda event: self.on_add_task())
self.entry.focus_set()

add_btn = Button(
    input_frame, text="+ Add Task", font=self.font_button,
    bg=COLOR_ACCENT, fg=COLOR_SIDEBAR, activebackground=COLOR_ACCENT_DARK,
    relief="flat", bd=0, padx=16, cursor="hand2",
    command=self.on_add_task
)
add_btn.pack(side=RIGHT, ipady=6)

def _build_task_list_area(self):
    list_frame = Frame(self.root, bg=COLOR_BG)
    list_frame.pack(fill=BOTH, expand=True, padx=20, pady=(0, 10))

    scrollbar = Scrollbar(list_frame, bg=COLOR_BG)
    scrollbar.pack(side=RIGHT, fill=Y)

    self.listbox = Listbox(
        list_frame, font=self.font_normal, bg=COLOR_CARD, fg=COLOR_TEXT,
        selectbackground=COLOR_ACCENT, selectforeground=COLOR_SIDEBAR,
        activestyle="none", relief="flat", bd=0,
        yscrollcommand=scrollbar.set, highlightthickness=0
    )
    self.listbox.pack(side=LEFT, fill=BOTH, expand=True)
    scrollbar.config(command=self.listbox.yview)

    self.listbox.bind("<Double-Button-1>", lambda event: self.on_toggle_done())

    action_frame = Frame(self.root, bg=COLOR_BG)
    action_frame.pack(fill=X, padx=20, pady=(0, 10))

    self._action_button(action_frame, "Mark Done / Undo", self.on_toggle_done,
COLOR_SUCCESS)
    self._action_button(action_frame, "Edit", self.on_edit_task, COLOR_ACCENT)
    self._action_button(action_frame, "Delete", self.on_delete_task, COLOR_DANGER)
    self._action_button(action_frame, "Clear Completed", self.on_clear_completed,
COLOR_TEXT_DIM)

def _action_button(self, parent, text, command, color):
    btn = Button(
        parent, text=text, font=self.font_small, bg=COLOR_CARD, fg=color,
        activebackground=COLOR_SIDEBAR, relief="flat", bd=0,
        padx=10, pady=6, cursor="hand2", command=command
    )
    btn.pack(side=LEFT, expand=True, fill=X, padx=4)
    return btn

def _build_footer(self):
    footer = Frame(self.root, bg=COLOR_SIDEBAR, height=40)
    footer.pack(fill=X, side=BOTTOM)

```

```

        self.status_var = StringVar()
        status_label = Label(
            footer, textvariable=self.status_var, font=self.font_small,
            fg=COLOR_TEXT_DIM, bg=COLOR_SIDEBAR
        )
        status_label.pack(pady=10)

# -----
# EVENT HANDLERS (CONTROLLER LOGIC)
# -----
def on_add_task(self):
    """Read text from the entry box and add it as a new task."""
    text = self.entry_var.get().strip()

    if not text:
        messagebox.showwarning("Empty Task", "Please type a task before adding it.")
        return

    self.manager.add_task(text)
    self.entry_var.set("")
    self.refresh_task_list()

def on_toggle_done(self):
    """Toggle the completed status of the selected task."""
    index = self._get_selected_index()
    if index is None:
        return
    self.manager.toggle_done(index)
    self.refresh_task_list()

def on_edit_task(self):
    """Edit the text of the selected task using a simple input dialog."""
    index = self._get_selected_index()
    if index is None:
        return

    current_text = self.manager.tasks[index]["text"]
    new_text = self._prompt_edit_dialog(current_text)

    if new_text is not None:
        new_text = new_text.strip()
        if new_text:
            self.manager.edit_task(index, new_text)
            self.refresh_task_list()
        else:
            messagebox.showwarning("Empty Task", "Task text cannot be empty.")

def on_delete_task(self):
    """Delete the selected task after confirmation."""
    index = self._get_selected_index()
    if index is None:
        return

    task_text = self.manager.tasks[index]["text"]
    confirm = messagebox.askyesno(
        "Confirm Delete",
        f"Delete this task?\n\n\"{task_text}\""
    )
    if confirm:
        self.manager.delete_task(index)
        self.refresh_task_list()

```

```

def on_clear_completed(self):
    """Remove all completed tasks after confirmation."""
    _, completed, _ = self.manager.counts()
    if completed == 0:
        messagebox.showinfo("Nothing to Clear", "There are no completed tasks to clear.")
        return

    confirm = messagebox.askyesno(
        "Clear Completed Tasks",
        f"Remove all {completed} completed task(s)?"
    )
    if confirm:
        self.manager.clear_completed()
        self.refresh_task_list()

# -----
# HELPER METHODS
# -----
def _get_selected_index(self):
    """Return the index of the selected listbox item, or None."""
    selection = self.listbox.curselection()
    if not selection:
        messagebox.showinfo("No Task Selected", "Please select a task first.")
        return None
    return selection[0]

def _prompt_edit_dialog(self, current_text):
    """
    A small custom dialog window for editing a task,
    using Tkinter's built-in simpledialog module.
    """
    from tkinter import simpledialog
    return simpledialog.askstring(
        "Edit Task", "Update task text:", initialvalue=current_text
    )

def refresh_task_list(self):
    """
    Redraw the entire listbox based on the current state
    of self.manager.tasks. Also updates the status bar.
    """
    self.listbox.delete(0, END)

    for task in self.manager.tasks:
        prefix = "[x] " if task["done"] else "[ ] "
        display_text = f"{prefix}{task['text']}"
        self.listbox.insert(END, display_text)

        # Color completed tasks differently
        index = self.listbox.size() - 1
        if task["done"]:
            self.listbox.itemconfig(index, fg=COLOR_DONE)
        else:
            self.listbox.itemconfig(index, fg=COLOR_TEXT)

    self._update_status()

def _update_status(self):
    total, completed, pending = self.manager.counts()
    self.status_var.set(

```

```

        f"Total: {total} | Pending: {pending} | Completed: {completed}"
        f" | Double-click a task to mark done/undo"
    )

def main():
    """Application entry point."""
    root = Tk()
    app = TodoApp(root)
    root.mainloop()

if __name__ == "__main__":
    main()

```

5. Detailed Walkthrough of Key Methods

5.1 TaskManager.add_task()

```

def add_task(self, text):
    task = {
        "text": text,
        "done": False,
        "created": datetime.now().strftime("%Y-%m-%d %H:%M")
    }
    self.tasks.append(task)
    self.save_tasks()

```

Line	Explanation
def add_task(self, text):	Defines a method on TaskManager that receives the new task's text
task = { ... }	Builds a dictionary representing one task: its text, completion status, and creation timestamp
self.tasks.append(task)	Adds this dictionary to the in-memory list of tasks
self.save_tasks()	Immediately writes the updated list to tasks.json so nothing is lost

5.2 TodoApp.on_add_task()

```

def on_add_task(self):
    text = self.entry_var.get().strip()

    if not text:
        messagebox.showwarning("Empty Task", "Please type a task before adding it.")
        return

    self.manager.add_task(text)
    self.entry_var.set("")
    self.refresh_task_list()

```

Line	Explanation
<code>text = self.entry_var.get().strip()</code>	Reads the current text typed in the input box and removes extra spaces
<code>if not text: ... return</code>	If the user typed nothing, show a warning popup and stop here
<code>self.manager.add_task(text)</code>	Delegates the actual data work to the TaskManager (Model)
<code>self.entry_var.set("")</code>	Clears the input box, ready for the next task
<code>self.refresh_task_list()</code>	Redraws the on-screen list so the new task appears immediately

5.3 TodoApp.refresh_task_list()

```
def refresh_task_list(self):
    self.listbox.delete(0, END)

    for task in self.manager.tasks:
        prefix = "[x] " if task["done"] else "[ ] "
        display_text = f"{prefix}{task['text']}"
        self.listbox.insert(END, display_text)

        index = self.listbox.size() - 1
        if task["done"]:
            self.listbox.itemconfig(index, fg=COLOR_DONE)
        else:
            self.listbox.itemconfig(index, fg=COLOR_TEXT)

    self._update_status()
```

Line	Explanation
<code>self.listbox.delete(0, END)</code>	Clears every item currently shown in the list box
<code>for task in self.manager.tasks:</code>	Loops through every task dictionary stored by TaskManager
<code>prefix = "[x] " if task["done"] else "[] "</code>	Chooses a checkbox-style marker depending on completion status
<code>self.listbox.insert(END, display_text)</code>	Adds the formatted task text to the bottom of the visible list
<code>self.listbox.itemconfig(index, fg=...)</code>	Changes the text color of that specific row — grey if done, normal color if pending
<code>self._update_status()</code>	Updates the Total/Pending/Completed counters shown at the bottom

6. Installation and Running Instructions

6.1 Prerequisites

- Python 3.8 or higher installed (Tkinter is included automatically on Windows and macOS)
- On Linux, if Tkinter is missing, install it with: `sudo apt-get install python3-tk`
- No pip installations are required — this project uses only the Python standard library

6.2 Running the Application

1. Save the source code as `todo_gui.py`
2. Open a terminal in the same folder as `todo_gui.py`
3. Run: `python todo_gui.py` (or `python3 todo_gui.py` on Linux/macOS)
4. The application window will open titled 'DecodeLabs To-Do List Manager'
5. A file named `tasks.json` will be automatically created in the same folder the first time you add a task

6.3 Using the Application

Action	How To Do It
Add a task	Type in the text box and press Enter, or click '+ Add Task'
Mark a task complete / undo	Select the task, then click 'Mark Done / Undo' (or double-click the task)
Edit a task	Select the task, click 'Edit', update the text in the popup, click OK
Delete a task	Select the task, click 'Delete', confirm in the popup
Remove all completed tasks	Click 'Clear Completed' and confirm
View statistics	Always visible at the bottom: Total, Pending, Completed counts

7. Testing the GUI Application

7.1 Test Cases

#	Test Scenario	Steps	Expected Result
1	Add a valid task	Type 'Finish Project 1', press Enter	Task appears in list as '[] Finish Project 1'; counters update
2	Add empty task	Leave box empty, click '+ Add Task'	Warning popup: 'Please type a task before adding it.'
3	Mark task as done	Select task, click 'Mark Done / Undo'	Marker changes to '[x]'; text turns grey; Completed count increases
4	Undo a completed task	Select a '[x]' task, click 'Mark Done / Undo' again	Marker changes back to '[]'; text returns to normal color
5	Edit a task	Select task, click 'Edit', change text, click OK	Task text updates in the list immediately
6	Cancel an edit	Select task, click 'Edit', click Cancel	No changes made to the task
7	Delete a task	Select task, click 'Delete', confirm Yes	Task removed from list; counters update
8	Delete with no selection	Click 'Delete' without selecting a task	Info popup: 'Please select a task first.'
9	Clear completed tasks	Mark 2 tasks done, click 'Clear Completed', confirm	Both completed tasks removed; pending tasks remain
10	Persistence test	Add tasks, close the app, reopen it	All previously added tasks reappear (loaded from tasks.json)

7.2 Edge Cases

- What if tasks.json becomes corrupted or contains invalid JSON? The `_load_tasks()` method catches `json.JSONDecodeError` and starts with an empty list instead of crashing.
- What if the user resizes the window very small? `minsize(440, 520)` prevents the window from becoming unusably small.
- What if the user double-clicks empty space in the list (no task there)? `curselection()` returns nothing, so `_get_selected_index()` safely shows an info popup instead of crashing.